

Module 3: Advanced SQL Queries

Q1. A query joins branch, customer, account, account_txn and card. How many join conditions does it need, and what happens if you supply one fewer?

Five tables in a simple chain need four join conditions. If one condition is missing, the disconnected tables can form a Cartesian product, producing duplicate combinations and inflated totals.

Q2. Explain the difference between INNER JOIN and LEFT JOIN in the two customer/account queries.

INNER JOIN returns only customers with at least one matching account. LEFT JOIN returns every customer and adds account information when a match exists, so it also keeps customers with no account.

Q3. A query was meant to list every branch with overdue loans, including branches with none. Explain the defect and write the fix.

The WHERE condition on the right table removes the NULL rows produced by the LEFT JOIN. Put the status condition inside the JOIN instead.

```
SELECT b.branch_name, l.loan_id
FROM branch b
LEFT JOIN loan l
  ON l.branch_id = b.branch_id
 AND l.status = 'OVERDUE';
```

Q4. Write the general shape of an anti-join and state which column must be tested for NULL.

Start with the rows you want, LEFT JOIN the rows that would match them, and keep only rows where the right-side match is NULL. Test a guaranteed non-null column from the right table, normally its primary key, because a nullable column could be NULL even when a matching row exists.

Q5. Why does the employee hierarchy query need a LEFT JOIN rather than an inner join?

The top-level Branch Manager can have manager_id = NULL. LEFT JOIN keeps that manager and shows a blank manager value; an inner join would remove the top-level manager.

Q6. For COUNT(*), COUNT(interest_rate), COUNT(DISTINCT account_type) and AVG(balance), say how NULLs are handled and what is expected for account.

COUNT(*) counts all 36 rows, including NULLs. COUNT(interest_rate) ignores NULL interest_rate values, so its exact value is the number of accounts where interest_rate is not NULL. COUNT(DISTINCT account_type) ignores NULL and counts the distinct account types. AVG(balance) ignores NULL balances and averages the non-NULL balances.

Q7. State the GROUP BY rule and explain why the engine cannot pick a value for an ungrouped column.

Every selected column must be in GROUP BY or be inside an aggregate function. The engine cannot arbitrarily choose an ungrouped value because one group may contain several different values.

Q8. Complete the WHERE/HAVING comparison.

WHERE filters individual rows before grouping and cannot normally use aggregate results. HAVING filters groups after GROUP BY and can use aggregates. Prefer WHERE for row-level conditions because it reduces rows before grouping; use HAVING for group-level conditions.

Q9. The NOT IN query returns zero rows even though customers are not nominees. Explain why and give two fixes.

If the subquery contains a NULL nominee_name, NOT IN becomes UNKNOWN for comparisons that are not equal to a known value, so no rows pass the WHERE condition. Fix it by filtering NULLs in the subquery or by using NOT EXISTS.

```
SELECT COUNT(*)
FROM customer c
WHERE c.first_name NOT IN (
    SELECT nominee_name
    FROM fixed_deposit
    WHERE nominee_name IS NOT NULL
);

SELECT COUNT(*)
FROM customer c
WHERE NOT EXISTS (
    SELECT 1
    FROM fixed_deposit f
    WHERE f.nominee_name = c.first_name
);
```

Q10. Explain fan-out and give a banking report where it produces a wrong but believable number.

Fan-out occurs when a parent matches several child rows and a later join multiplies those matches. For example, joining customers to accounts and then transactions can repeat an account balance once per transaction. SUM(balance) can therefore look believable while being too high.

Q11. List every card with account number, customer full name, branch name, card type, masked number and status. Sort by branch and customer.

Use card → account → customer and account → branch joins. The query below returns the requested columns.

```
SELECT c.card_id,
       a.account_number,
       cu.first_name || ' ' || cu.last_name AS customer_name,
       b.branch_name,
       c.card_type,
       c.masked_number,
       c.status
FROM card c
JOIN account a ON a.account_id=c.account_id
JOIN customer cu ON cu.customer_id=a.customer_id
JOIN branch b ON b.branch_id=a.branch_id
ORDER BY b.branch_name, customer_name;
```

Q12. List all customers at Koramangala and Whitefield with account numbers and balances, richest first. Include customers with no account.

Use LEFT JOIN from customer to account so customers without an account remain in the result.

```
SELECT cu.customer_id,
       cu.first_name || ' ' || cu.last_name AS customer_name,
       b.branch_name,
       a.account_number,
       a.balance
FROM customer cu
JOIN branch b ON b.branch_id=cu.home_branch_id
LEFT JOIN account a ON a.customer_id=cu.customer_id
WHERE b.branch_name IN ('Koramangala','Whitefield')
ORDER BY a.balance DESC NULLS LAST, customer_name;
```

Q13. Find every account that has never had a transaction, showing account number, type, status and customer name.

NOT EXISTS is suitable because we want accounts for which no matching transaction exists.

```
SELECT a.account_number,
       a.account_type,
       a.status,
       cu.first_name || ' ' || cu.last_name AS customer_name
FROM account a
JOIN customer cu ON cu.customer_id=a.customer_id
WHERE NOT EXISTS (
    SELECT 1 FROM account_txn t
    WHERE t.account_id=a.account_id
)
ORDER BY a.account_number;
```

Q14. Find every account that has no card issued against it. Show account number, type and customer name. Give the count and list.

Use an anti-join with NOT EXISTS. The first query gives the list and the second gives its count.

```

SELECT a.account_number,a.account_type,
       cu.first_name || ' ' || cu.last_name AS customer_name
FROM account a
JOIN customer cu ON cu.customer_id=a.customer_id
WHERE NOT EXISTS (
    SELECT 1 FROM card c WHERE c.account_id=a.account_id
)
ORDER BY a.account_number;

SELECT COUNT(*) AS accounts_without_card
FROM account a
WHERE NOT EXISTS (
    SELECT 1 FROM card c WHERE c.account_id=a.account_id
);

```

Q15. Produce the full staff list showing employee, designation, branch and manager. Branch Managers must appear.

The manager relationship is optional, so use LEFT JOIN from employee to employee.

```

SELECT e.first_name || ' ' || e.last_name AS employee_name,
       e.designation,
       b.branch_name,
       COALESCE(m.first_name || ' ' || m.last_name,'No manager') AS
manager_name
FROM employee e
JOIN branch b ON b.branch_id=e.branch_id
LEFT JOIN employee m ON m.employee_id=e.manager_id
ORDER BY b.branch_name, employee_name;

```

Q16. For each Branch Manager, show name, branch, team size and total team salary excluding their own.

Join employees to their manager and group by the manager. Because the manager is not joined to themselves through manager_id, the salary sum excludes the manager.

```

SELECT m.first_name || ' ' || m.last_name AS manager_name,
       b.branch_name,
       COUNT(e.employee_id) AS team_size,
       COALESCE(SUM(e.salary),0) AS team_salary
FROM employee m
JOIN branch b ON b.branch_id=m.branch_id
LEFT JOIN employee e ON e.manager_id=m.employee_id
WHERE m.designation='Branch Manager'
GROUP BY m.employee_id,m.first_name,m.last_name,b.branch_name
ORDER BY b.branch_name;

```

Q17. Build loan_risk_band and use a non-equi join to report loan count and outstanding in each band.

The join uses >= and < rather than equality, so each loan is placed into the range that contains its outstanding amount.

```

CREATE TABLE loan_risk_band (
    band_code VARCHAR(2) PRIMARY KEY,
    band_name VARCHAR(20) NOT NULL,
    min_outstanding NUMERIC(18,2) NOT NULL,
    max_outstanding NUMERIC(18,2) NOT NULL
);

INSERT INTO loan_risk_band VALUES
('L1','Small',0,500000),
('L2','Medium',500000,2000000),
('L3','Large',2000000,5000000),
('L4','Very Large',5000000,99999999);

SELECT r.band_code,r.band_name,
       COUNT(l.loan_id) AS loan_count,
       COALESCE(SUM(l.outstanding),0) AS total_outstanding
FROM loan_risk_band r
LEFT JOIN loan l
    ON l.outstanding>=r.min_outstanding
   AND l.outstanding<r.max_outstanding
GROUP BY r.band_code,r.band_name
ORDER BY r.band_code;

```

Q18. For each branch, report customer count, account count, total balance and average balance. Include branches with no customers.

Start from branch and use LEFT JOIN so branches with no customers remain.

```

SELECT b.branch_name,
       COUNT(DISTINCT cu.customer_id) AS customer_count,
       COUNT(DISTINCT a.account_id) AS account_count,
       COALESCE(SUM(a.balance),0) AS total_balance,
       AVG(a.balance) AS average_balance
FROM branch b
LEFT JOIN customer cu ON cu.home_branch_id=b.branch_id
LEFT JOIN account a ON a.customer_id=cu.customer_id
GROUP BY b.branch_id,b.branch_name
ORDER BY total_balance DESC;

```

Q19. For each loan type, report loan count, total principal, total outstanding and percentage outstanding.

Aggregate by loan_type and divide total outstanding by total principal. NULLIF prevents division by zero.

```

SELECT loan_type,
       COUNT(*) AS loan_count,
       SUM(principal_amount) AS total_principal,
       SUM(outstanding) AS total_outstanding,
       ROUND(100.0*SUM(outstanding)/NULLIF(SUM(principal_amount),0),1)
       AS outstanding_percentage
FROM loan
GROUP BY loan_type
ORDER BY loan_type;

```

Q20. For each channel, report successful/failed counts, successful value and failure rate.

Use conditional aggregation so each channel produces one row.

```
SELECT channel,
       COUNT(*) FILTER(WHERE status='SUCCESS') AS successful_transactions,
       COUNT(*) FILTER(WHERE status='FAILED') AS failed_transactions,
       COALESCE(SUM(amount) FILTER(WHERE status='SUCCESS'),0) AS
successful_value,
       ROUND(100.0*COUNT(*) FILTER(WHERE status='FAILED')/NULLIF(COUNT(*),0),2)
       AS failure_rate
FROM account_txn
GROUP BY channel
ORDER BY failure_rate DESC;
```

Q21. Find every branch whose total outstanding loan book exceeds 2,000,000.

Use HAVING because the condition is on an aggregate value.

```
SELECT b.branch_name,
       COUNT(l.loan_id) AS loan_count,
       SUM(l.outstanding) AS total_outstanding
FROM branch b
JOIN loan l ON l.branch_id=b.branch_id
GROUP BY b.branch_id,b.branch_name
HAVING SUM(l.outstanding)>2000000
ORDER BY total_outstanding DESC;
```

Q22. For each customer holding at least two accounts, show name, account count and combined balance.

Group by customer and keep only groups with at least two accounts.

```
SELECT cu.customer_id,
       cu.first_name || ' ' || cu.last_name AS customer_name,
       COUNT(a.account_id) AS account_count,
       SUM(a.balance) AS combined_balance
FROM customer cu
JOIN account a ON a.customer_id=cu.customer_id
GROUP BY cu.customer_id,cu.first_name,cu.last_name
HAVING COUNT(a.account_id)>=2
ORDER BY combined_balance DESC;
```

Q23. Produce a monthly transaction summary with transaction count, credits, debits and net movement.

Group transactions by the month extracted from txn_date.

```
SELECT DATE_TRUNC('month',txn_date)::date AS month,
       COUNT(*) AS transaction_count,
       COALESCE(SUM(amount) FILTER(WHERE txn_type='CREDIT'),0) AS
total_credits,
       COALESCE(SUM(amount) FILTER(WHERE txn_type='DEBIT'),0) AS total_debits,
       COALESCE(SUM(CASE
```

```

        WHEN txn_type='CREDIT' THEN amount
        WHEN txn_type='DEBIT' THEN -amount
        ELSE 0 END),0) AS net_movement
FROM account_txn
GROUP BY DATE_TRUNC('month',txn_date)
ORDER BY month;

```

Q24. List every account whose balance is more than twice the bank-wide average balance.

Calculate the average once in a scalar subquery and compare every account with it.

```

SELECT account_number,balance
FROM account
WHERE balance > 2*(SELECT AVG(balance) FROM account)
ORDER BY balance DESC;

```

Q25. For each branch, show the balance of its single largest account using a correlated subquery.

For each account, the subquery finds the maximum balance within that account's branch. Only accounts equal to that maximum are returned.

```

SELECT b.branch_name,a.account_number,a.balance
FROM account a
JOIN branch b ON b.branch_id=a.branch_id
WHERE a.balance=(
    SELECT MAX(a2.balance)
    FROM account a2
    WHERE a2.branch_id=a.branch_id
)
ORDER BY b.branch_name;

```

Q26. List customers who hold a loan but no fixed deposit using NOT EXISTS and LEFT JOIN anti-join.

The first query checks that a loan exists and a fixed deposit does not. The second creates the same logic with a LEFT JOIN and a NULL test.

```

SELECT cu.customer_id,
       cu.first_name || ' ' || cu.last_name AS customer_name
FROM customer cu
WHERE EXISTS(SELECT 1 FROM loan l WHERE l.customer_id=cu.customer_id)
AND NOT EXISTS(SELECT 1 FROM fixed_deposit f WHERE
f.customer_id=cu.customer_id);

SELECT DISTINCT cu.customer_id,
       cu.first_name || ' ' || cu.last_name AS customer_name
FROM customer cu
JOIN loan l ON l.customer_id=cu.customer_id
LEFT JOIN fixed_deposit f ON f.customer_id=cu.customer_id
WHERE f.customer_id IS NULL;

```

Q27. Find customers whose total transaction value this quarter is above the average total across all customers. Use a derived table.

First calculate one total per customer in a derived table. Then compare those totals with the average of the derived table.

```
WITH customer_totals AS (  
    SELECT cu.customer_id,  
           COALESCE(SUM(t.amount),0) AS total_value  
    FROM customer cu  
    LEFT JOIN account a ON a.customer_id=cu.customer_id  
    LEFT JOIN account_txn t ON t.account_id=a.account_id  
        AND t.txn_date>=DATE_TRUNC('quarter',CURRENT_DATE)  
        AND t.txn_date<DATE_TRUNC('quarter',CURRENT_DATE)+INTERVAL '3 months'  
    GROUP BY cu.customer_id  
)  
SELECT customer_id,total_value  
FROM customer_totals  
WHERE total_value>(SELECT AVG(total_value) FROM customer_totals);
```

Q28. Produce one list of every ICICI Bank product holding from fixed_deposit, loan and account using set operators.

UNION ALL combines the three product sources into one result without removing separate holdings.

```
SELECT customer_id,'FIXED_DEPOSIT' AS product_type,principal_amount AS amount  
FROM fixed_deposit  
UNION ALL  
SELECT customer_id,'LOAN',principal_amount  
FROM loan  
UNION ALL  
SELECT customer_id,account_type,balance  
FROM account  
ORDER BY customer_id,product_type;
```

Q29. Using set operators, find customers holding both a deposit and a loan, deposit but no loan, and loan but no deposit.

INTERSECT finds the common customers. EXCEPT removes customers appearing in the other set. The three counts partition customers who hold at least one of the two products.

```
WITH deposits AS (SELECT customer_id FROM fixed_deposit),  
loans AS (SELECT customer_id FROM loan),  
both_products AS (  
    SELECT customer_id FROM deposits  
    INTERSECT  
    SELECT customer_id FROM loans  
) ,  
deposit_only AS (  
    SELECT customer_id FROM deposits  
    EXCEPT  
    SELECT customer_id FROM loans  
) ,  
loan_only AS (  
    SELECT customer_id FROM loans  
    EXCEPT
```

```

        SELECT customer_id FROM deposits
    )
    SELECT 'BOTH' AS category, COUNT(*) AS customer_count FROM both_products
    UNION ALL
    SELECT 'DEPOSIT_ONLY', COUNT(*) FROM deposit_only
    UNION ALL
    SELECT 'LOAN_ONLY', COUNT(*) FROM loan_only;

```

Q30. Identify every defect in the monthly-pack query and rewrite it correctly.

The query has fan-out because account balances are joined to transaction rows before aggregation. Therefore SUM(a.balance) can be inflated. COUNT(c.customer_id) can also count the same customer more than once. A half-open date range is safer for timestamp columns than BETWEEN through March 31. Aggregate transactions per account first, then join that result to the account/customer data.

```

WITH txn_by_account AS (
    SELECT account_id, SUM(amount) AS total_transacted
    FROM account_txn
    WHERE txn_date >= DATE '2026-01-01'
    AND txn_date < DATE '2026-04-01'
    GROUP BY account_id
)
SELECT b.branch_name,
    COUNT(DISTINCT cu.customer_id) AS customers,
    SUM(a.balance) AS total_deposits,
    COALESCE(SUM(t.total_transacted), 0) AS total_transacted
FROM branch b
JOIN customer cu ON cu.home_branch_id = b.branch_id
JOIN account a ON a.customer_id = cu.customer_id
LEFT JOIN txn_by_account t ON t.account_id = a.account_id
GROUP BY b.branch_id, b.branch_name;

```

Q31. Why cannot GROUP BY and HAVING alone answer top three customers by balance in each branch?

GROUP BY produces one result per group, but top three requires ranking individual customers inside each branch. A window function such as ROW_NUMBER or DENSE_RANK with PARTITION BY branch is needed. The programme covers window functions in Module 5.

Q32. A NOT IN compliance report has returned zero rows for six weeks. What does this tell you about designing checks for reports whose correct answer is often zero?

A zero result is not automatically proof that there are no exceptions. A NULL in a NOT IN subquery can make the predicate UNKNOWN and hide every exception. Exception reports should be tested with known positive cases as well as normal zero-result cases. Monitoring should also detect unexpected changes in result counts and data-quality conditions.

Q33. The branch performance pack takes 40 seconds against 40 million transaction rows. Without changing SQL, list three things you would do and name the module.

Review or add indexes for joins and filters, examine the execution plan and database performance, and consider a maintained summary/reporting table instead of repeatedly scanning the full transaction history. Index and query-performance work is covered in Module 5; deliberate summary/denormalization is introduced in Module 4 and used in later reporting modules.

Q34. Prove every transaction in the acquired-bank extract landed in ICICI and that ICICI gained none that was not in the extract.

Use EXCEPT in both directions. Extract EXCEPT core shows transactions missing from ICICI. Core EXCEPT extract shows transactions in ICICI that were not in the extract. Both should be empty for an exact match. A difference may also be caused by a legitimate transformation such as a changed identifier or normalized value.

```
SELECT transaction_id,account_id,txn_date,amount,txn_type
FROM acquired_bank_extract
EXCEPT
SELECT transaction_id,account_id,txn_date,amount,txn_type
FROM account_txn;
```

```
SELECT transaction_id,account_id,txn_date,amount,txn_type
FROM account_txn
EXCEPT
SELECT transaction_id,account_id,txn_date,amount,txn_type
FROM acquired_bank_extract;
```

Q35. Reply to ticket ICICI-4530 in 10–15 lines.

The monthly branch performance pack now shows customer counts, account balances and transaction activity by branch. The query separates transaction aggregation from account aggregation so join fan-out does not inflate totals. One finding is that transaction activity differs between branches. Another finding is that customer and account counts do not necessarily move together with transaction value. The pack should be treated as a summary report, not as a transaction-level audit. It also depends on the completeness and correctness of the source transaction data. Individual operational decisions should therefore be checked against the underlying records.